

RAPPORT TECHNIQUE FINAL

Livable L4

CocoaSecure IoT Platform

Monitoring temps réel du séchage du cacao — Détection de fraude sur données capteurs IoT

Établissement	Université Félix Houphouët-Boigny — UFR Mathématiques et Informatique
Filière	Master 1 BD-Génie Logiciel — 2025-2026
Équipe	Komoe edgar -Anon Sebastien-Kouadio Hippolyte
Enseignant	Dr. N'Golo KONATÉ, Maître-Assistant CAMES
Dépôt	https://github.com/leconcret/cocoasecure-iot-platform.git

1. Contexte métier ivoirien et problématique

La Côte d'Ivoire est le premier producteur mondial de cacao, filière stratégique pour l'économie nationale. La qualité des fèves dépend directement du contrôle des conditions de séchage — température, humidité, concentration en CO2 — sur les aires de coopératives. Une falsification des mesures issues des capteurs IoT peut conduire à la validation de lots non conformes, avec un impact direct sur la qualité export et la traçabilité de la filière.

CocoaSecure IoT Platform supervise en temps réel quatre coopératives ivoiriennes — San-Pedro Nord (ESP01), Soubré (ESP02), Grand-Béréby (ESP03) et Daloa (ESP04) — via des capteurs ESP32 publiant température, humidité, CO2, luminosité, niveau de batterie et qualité de signal sur un broker MQTT.

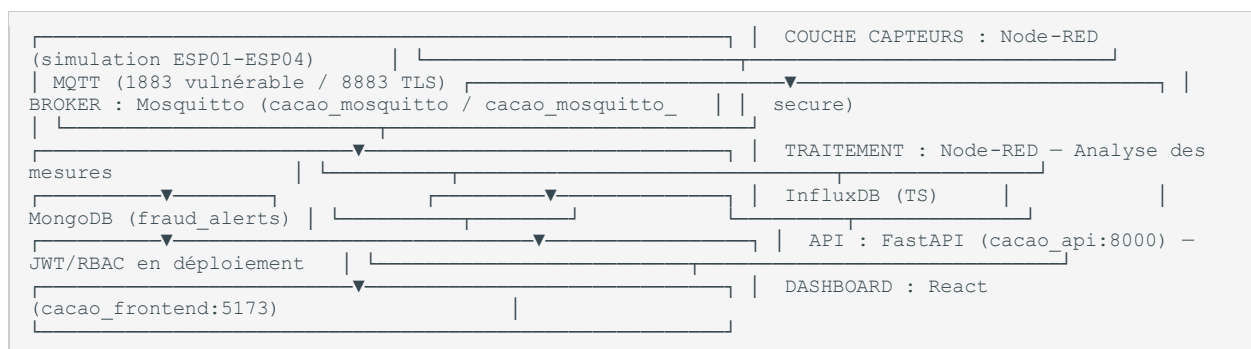
Problématique : comment détecter et prévenir la falsification des données de séchage, tout en garantissant la disponibilité et la performance de la plateforme de supervision face à une charge croissante de capteurs ?

2. Architecture IoT distribuée

La plateforme suit un pipeline en couches : capteurs ESP32 (simulés via Node-RED) publiant sur un broker MQTT Mosquitto, traités par un flow Node-RED qui valide les seuils métier et route les données vers InfluxDB (séries temporelles) et MongoDB (alertes de fraude), exposées via une API FastAPI consommée par un dashboard React.

Couche	Composants déployés
Simulation	Node-RED (flow "horodatage" → génération aléatoire 4 sites, ~25% de fraude simulée)
Broker MQTT	Mosquitto 2 — cacao_mosquitto (port 1883, vulnérable) + cacao_mosquitto_secure (port 8883, TLS 1.3 + mTLS)
Traitement	Node-RED — noeud "Analyse des mesures" (validation seuils), routage InfluxDB/MongoDB, alertes fraude
Stockage	InfluxDB 2.7 (bucket cacao_measurements, mesure sensor_measurements) + MongoDB 6 (collection fraud_alerts)
Cache	Redis 7-alpine — instance unique déployée (voir limites section 6)
API	FastAPI (cacao_api, port 8000) — endpoints /dashboard, /readings, /alerts ; JWT/RBAC en cours de déploiement
Frontend	React 18 + Vite (cacao_frontend, port 5173) — dashboard temps réel
Observabilité	Grafana (cacao_grafana, port 3000) — intégration en cours de stabilisation
Déploiement	Docker Compose — 9 services orchestrés (docker-compose up)

Schéma de principe de l'architecture en couches (vue déploiement) :



Justification des choix : l'architecture sépare strictement la couche d'ingestion (MQTT/Node-RED) de la couche de persistance (InfluxDB pour les séries temporelles à forte fréquence, MongoDB pour les documents d'alerte peu fréquents mais riches en métadonnées), ce qui permet de faire évoluer chaque composant indépendamment et de dupliquer le broker (vulnérable/sécurisé) sans impacter le reste du pipeline.

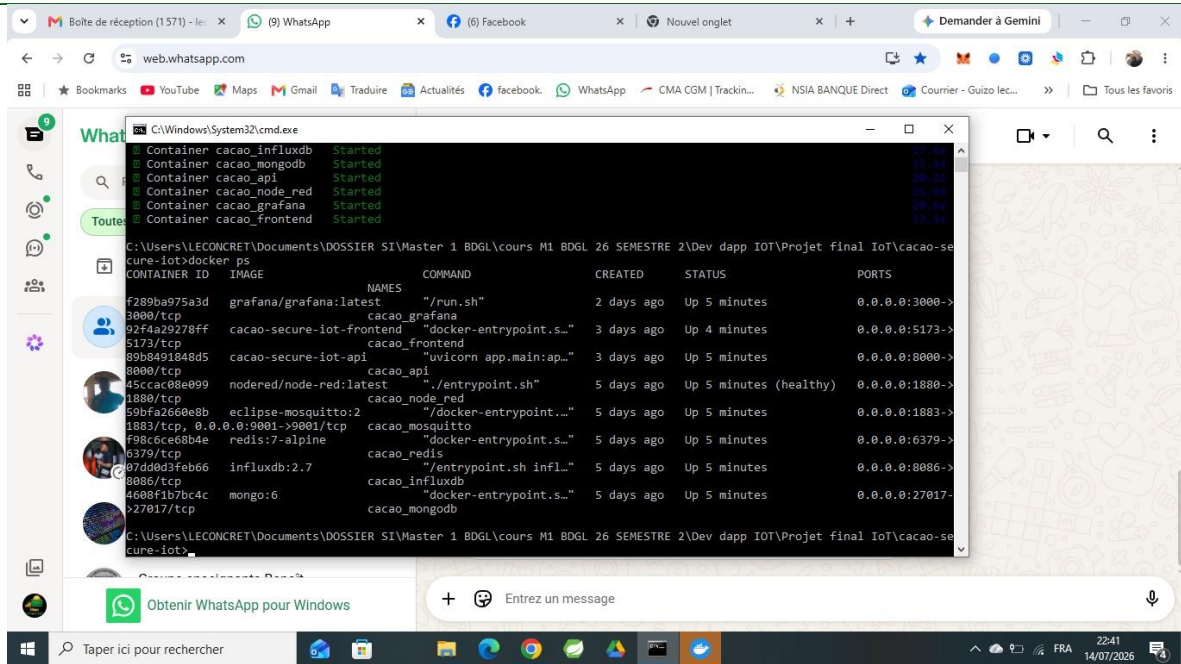


Figure 2 — Vue terminal docker ps confirmant le démarrage des 8 services principaux avec leurs ports mappés.

3. Analyse STRIDE

L'analyse de risques STRIDE a été menée sur les composants exposés du pipeline, avec un focus particulier sur le broker MQTT (point d'entrée principal des données capteurs) et l'API REST.

Menace STRIDE	Composant	Constat	Contre-mesure
Spoofing	Broker MQTT	Client peut usurper l'identité d'un capteur (deviceId arbitraire)	Auth mutuelle X.509 (CN = identité)
Tampering	Payload MQTT	Injection de valeurs falsifiées acceptée sans contrôle	TLS 1.3 + validation seuils Node-RED
Repudiation	API / logs	Traçabilité limitée des accès API sans authentification	JWT + logs horodatés (en déploiement)
Info. Disclosure	Canal MQTT	Payload JSON intégralement lisible en clair (confirmé Wireshark)	TLS 1.3 obligatoire
DoS	Broker / API	Aucune limite de connexions/débit sur la config initiale	max_connections Mosquitto + rate limiting API (slowapi)
Elevation Privilege	API	Aucune notion de rôle sur les endpoints actuels	RBAC 3 niveaux (viewer/operator/admin)

4. Rapport d'attaques — vecteurs, preuves, indicateurs de compromission

4.1 A1 — Sniffing MQTT

Interception passive du trafic MQTT en clair (port 1883) via Wireshark, filtre tcp.port==1883. Le payload JSON complet (deviceId, siteId, site, lotId, mesures) est extrait sans aucun effort de déchiffrement, confirmant la vulnérabilité Information Disclosure de la matrice STRIDE.

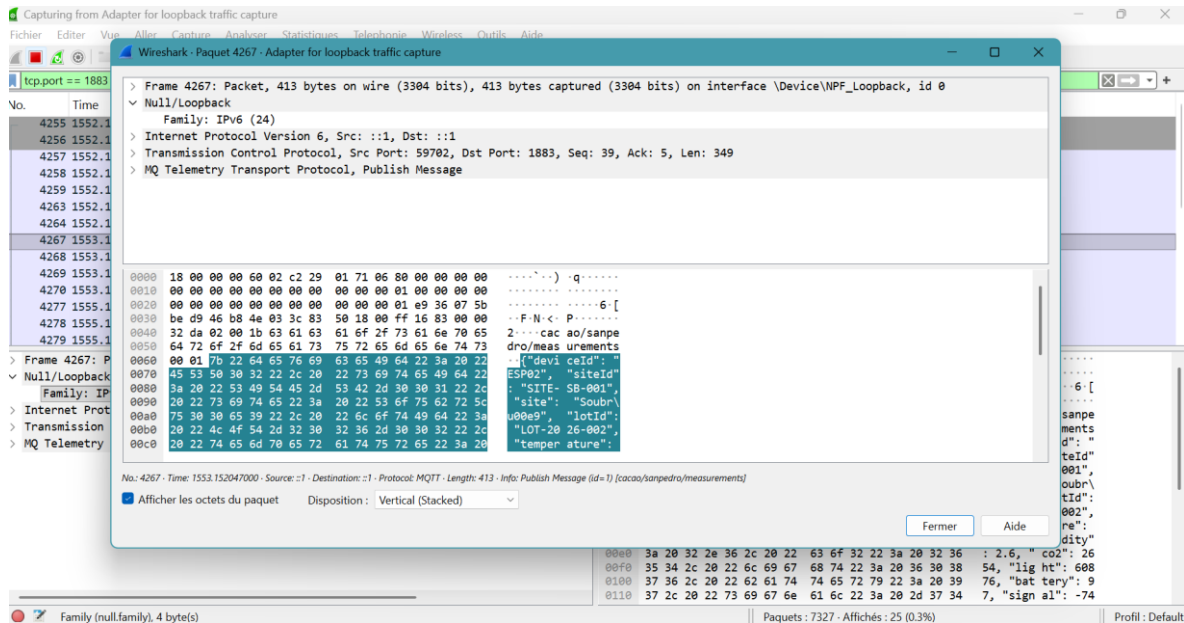


Figure 3 — Payload JSON intégralement lisible en clair, capteur ESP02 (Soubre).

4.2 A2 — Injection / usurpation de capteur

Un script Python (paho-mqtt) se connecte au broker sans identifiant et publie des mesures frauduleuses en usurpant l'identité deviceId=ESP01/ESP02/ESP04, avec des températures jusqu'à 92.9°C (plage normale : 34-40°C).

4.3 A3 — Replay

Un message légitime d'ESP01 est capturé puis rejoué à l'identique 3 fois. Le pipeline, ne disposant que d'une validation par seuils métier, accepte chaque rejeu comme une nouvelle mesure valide — limite documentée en section 9.

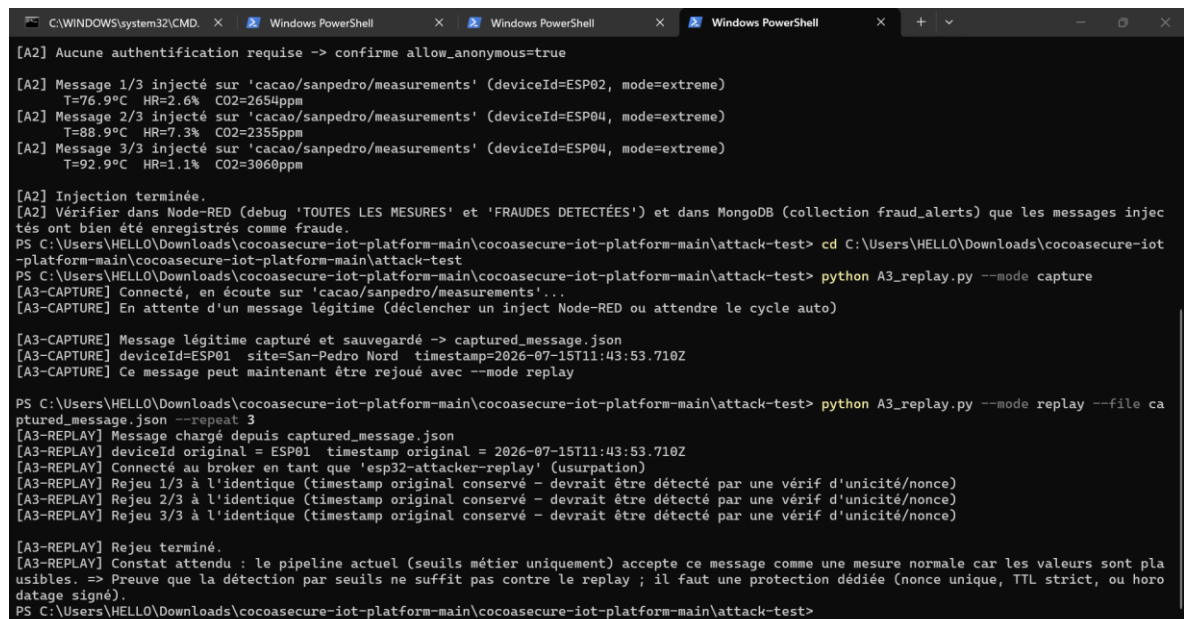


Figure 4 — Exécution des attaques A2 (injection, T° jusqu'à 92.9°C) et A3 (capture + rejeu x3) contre le broker vulnérable.

Indicateurs de compromission (IoC) identifiés

deviceId publiant hors de sa fenêtre temporelle habituelle ; valeurs hors seuils physiques plausibles ($T^\circ > 60^\circ\text{C}$) ; absence de certificat client sur le port sécurisé ; connexions anonymes multiples sur un intervalle court ($< 5\text{s}$).

5. Contre-mesures — TLS 1.3 et authentification mutuelle

Une PKI locale a été déployée : autorité de certification (CA) auto-signée émettant un certificat serveur (CN=mosquitto) et un certificat client par capteur (CN=esp01 à esp04), avec extensions X.509 extendedKeyUsage appropriées. Un second listener Mosquitto (port 8883) impose TLS 1.3 et exige un certificat client valide (require_certificate=true, allow_anonymous=false).

Critère	Avant (port 1883)	Après (port 8883)
Chiffrement	Aucun	TLS 1.3 / TLS_AES_256_GCM_SHA384
Authentification	allow_anonymous=true	Certificat X.509 obligatoire
Test A2 rejoué	Acceptée	Rejetée ("tls1 alert unknown ca")

Preuve — connexion avec certificat valide

New client connected ... (u'esp01'). Client negotiated TLSv1.3 cipher TLS_AES_256_GCM_SHA384. Le broker authentifie automatiquement le client via le CN du certificat, sans mot de passe séparé.

Preuve — connexion sans certificat

OpenSSL Error: error:0A000418:SSL routines::tls1 alert unknown ca. Client disconnected: Protocol error. La connexion est rejetée avant tout examen du contenu du message.

6. Architecture Redis et haute disponibilité

⚠ Limite identifiée — non-conformité au cahier des charges

Le déploiement actuel utilise une instance Redis unique (redis:7-alpine) sans configuration Sentinel. Le sujet exige un déploiement Redis Sentinel à 3 nœuds (1 master + 2 replicas + 3 sentinel) pour garantir le failover automatique. Cette non-conformité impacte directement le critère C3 de la grille d'évaluation, qui plafonne la note à 1.5/5 en l'absence de HA Redis. La correction de ce point est identifiée comme priorité immédiate pour l'équipe (section 9 — perspectives).

La configuration actuelle du service Redis, extraite du docker-compose.yml :

```
redis:  image: redis:7-alpine  container_name: cacao_redis  ports:      - "6379:6379"
restart: unless-stopped
```

Patterns Redis prévus dans l'architecture cible (à finaliser) : cache-aside pour la dernière mesure par capteur (TTL 30s), Pub/Sub pour la diffusion temps réel des alertes vers le dashboard, rate limiting sur l'API (100 req/min par token).

7. Pipeline de données

Le flow Node-RED orchestre l'ensemble du pipeline : un nœud "horodatage" déclenche la génération de mesures (aléatoirement normales ou frauduleuses, ~25% de fraude simulée) pour l'un des 4 sites, publiées sur le topic cacao/sanpedro/measurements. Le nœud "Analyse des mesures" applique les règles de seuils métier et route vers InfluxDB (mesures normales) ou MongoDB (alertes de fraude).

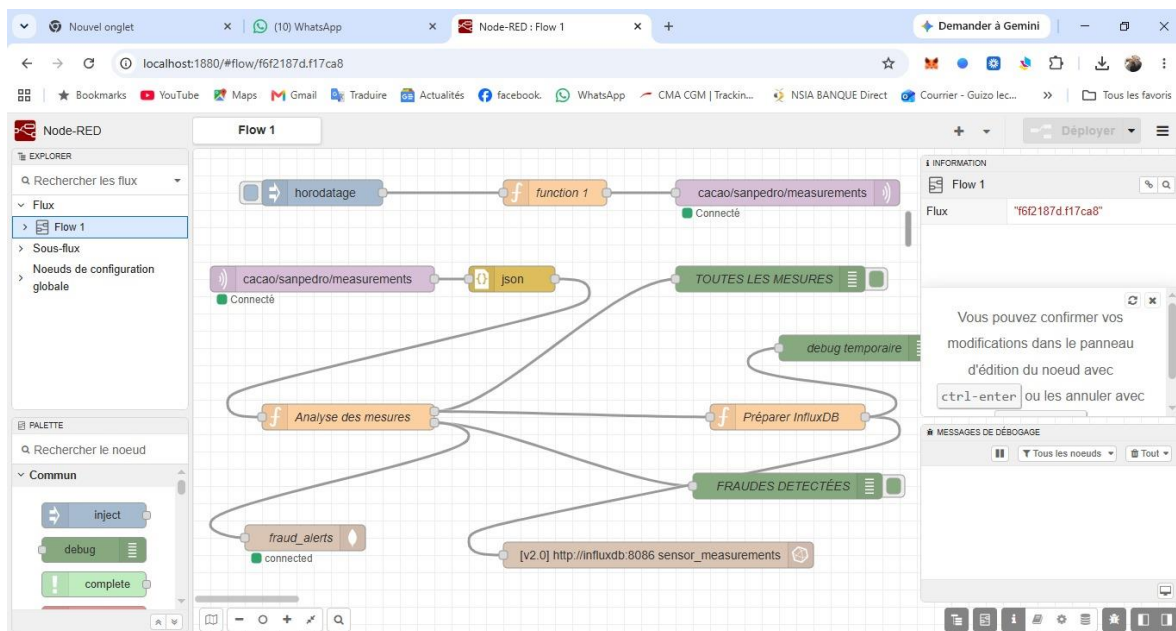


Figure 5 — Flow Node-RED complet : simulation, parsing JSON, analyse des mesures, routage InfluxDB/MongoDB, alertes fraude.

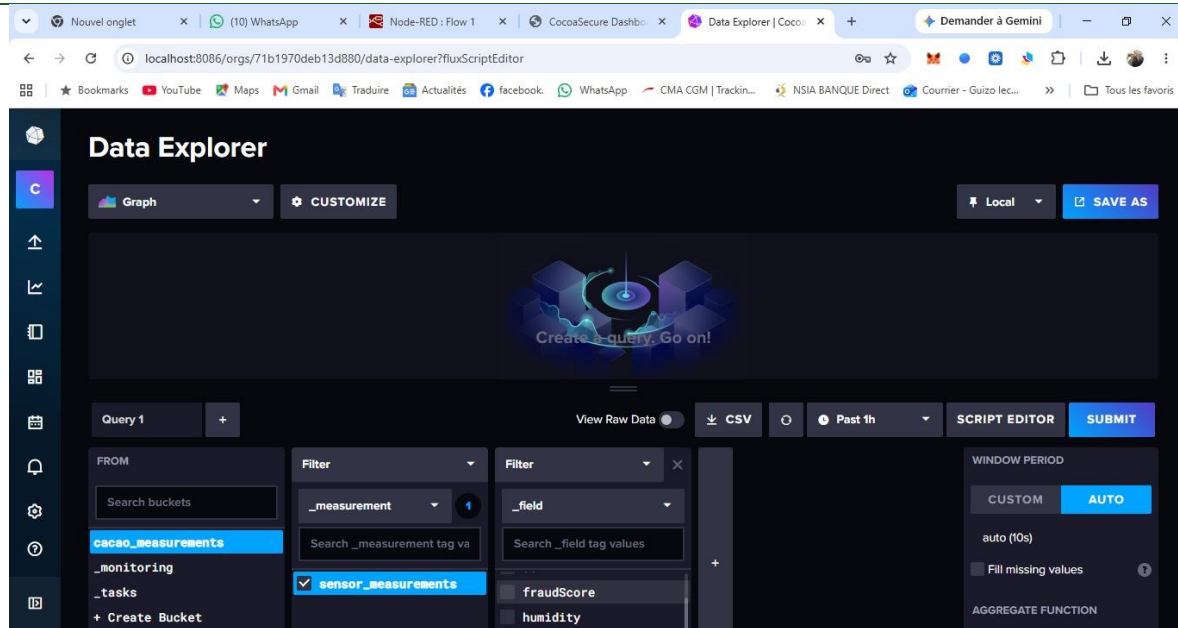


Figure 6 — InfluxDB Data Explorer, requête sur le bucket `cacao_measurements` / `sensor_measurements` avec les champs `battery`, `co2`, `fraudScore`, `humidity`.

Persistance des alertes de fraude — MongoDB

La collection `fraud_alerts` de la base `cocoasecure` stocke un document complet par alerte détectée, incluant le score de fraude, la liste des incohérences relevées et la mesure brute à l'origine de l'alerte. Une interrogation directe via `mongosh` confirme la structure et le contenu réel des données persistées :

```

C:\Users\LECONCRET\Documents\DOSSIER SI\Master 1 BDGL\cours M1 BDGL 26 SEMESTRE 2\Dev dapp IOT\Projet final IOT\cacao-secure-iot>docker exec -it cacao_mongodb mongosh
Current Mongosh Log ID: 6a5a4d5d3760dc72c99df8a2
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 6.0.28
Using Mongosh: 2.8.3

-----
The server generated these startup warnings when booting:
2026-07-17T15:14:30.588+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-07-17T15:14:30.554+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-07-17T15:14:30.554+00:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never' in this binary version
2026-07-17T15:14:30.554+00:00: vm.max_map_count is too low
-----

test> use cocoasecure
switched to db cocoasecure
cocoasecure> show collections
fraud_alerts
cocoasecure> db.fraud_alerts.find().pretty()
[
  {
    "_id": ObjectId("6a4a566fe67a870006668436"),
    "type": "FRAUDE",
    "niveau": "CRITICAL",
    "deviceId": "ESP03",
    "siteId": "SITE-00-001",
    "site": "Grand-Béréby",
    "lotId": "LOT-2026-003",
    "fraudScore": 90,
    "alerts": [
      "Température incohérente",
      "Humidité incohérente",
      "CO2 incohérent",
      "Batterie très faible"
    ],
    "mesure": {
      "deviceId": "ESP03",
      "siteId": "SITE-00-001",
      "site": "Grand-Béréby",
      "lotId": "LOT-2026-003",
      "temperature": 95,
      "humidity": 4,
      "co2": 2500
    }
  }
]

```

Figure 6d — Requête `db.fraud_alerts.find().pretty()` : alerte **CRITICAL** sur **ESP03** (Grand-Béréby), `fraudScore=90`, avec incohérences température/humidité/CO2/batterie détectées simultanément.



```

{
  "_id": ObjectId("6a4a566fe67a870006668436"),
  "type": "FRAUDE",
  "niveau": "CRITICAL",
  "deviceId": "ESP03",
  "siteId": "SITE-GB-001",
  "lotId": "LOT-2026-003",
  "fraudScore": 90,
  "alerts": [
    "Température incohérente",
    "Humidité incohérente",
    "CO2 incohérent",
    "Batterie très faible"
  ],
  "mesure": {
    "deviceId": "ESP03",
    "siteId": "SITE-GB-001",
    "lotId": "LOT-2026-003",
    "temperature": 95,
    "humidity": 4,
    "co2": 2500,
    "light": 1000,
    "battery": 3,
    "signal": -92,
    "timestamp": "2026-07-05T13:04:47.599Z",
    "fraudScore": 90,
    "isFraud": true,
    "alerts": [
      "Température incohérente",
      "Humidité incohérente",
      "CO2 incohérent",
      "Batterie très faible"
    ]
  },
  "createdAt": "2026-07-05T13:04:47.617Z"
},
{
  "_id": ObjectId("6a4a5fd8e67a870006668437"),
  "type": "FRAUDE",
  "niveau": "CRITICAL",
  "deviceId": "ESP03",
  "siteId": "SITE-GB-001",
  "lotId": "LOT-2026-003",
  "fraudScore": 90,
  "alerts": [
    "Température incohérente",
    "Humidité incohérente",
    "CO2 incohérent",
    "Batterie très faible"
  ],
  "mesure": {
    "deviceId": "ESP03",
    "siteId": "SITE-GB-001",
    "lotId": "LOT-2026-003",
    "temperature": 95,
    "humidity": 4,
    "co2": 2500,
    "light": 1000,
    "battery": 3,
    "signal": -92,
    "timestamp": "2026-07-05T13:04:47.599Z",
    "fraudScore": 90,
    "isFraud": true,
    "alerts": [
      "Température incohérente",
      "Humidité incohérente",
      "CO2 incohérent",
      "Batterie très faible"
    ]
  },
  "createdAt": "2026-07-05T13:04:47.617Z"
}

```

Figure 6e — Détail du document : champ mesure imbriqué (valeurs brutes du capteur), isFraud=true, horodatage createdAt distinct du timestamp de mesure — permettant de distinguer l'heure de détection de l'heure de relevé.

Cette structure documentaire confirme que le pipeline ne se contente pas de rejeter les mesures suspectes : il conserve l'intégralité du contexte (mesure originale + alertes déclenchées + score) nécessaire à une investigation a posteriori, répondant à l'exigence de traçabilité (catégorie Repudiation de la matrice STRIDE, section 3).

8. API REST

L'API FastAPI (cacao_api, port 8000) expose les endpoints `/dashboard`, `/readings/latest`, `/readings/history` et `/alerts`, consommés par le dashboard React. Une couche d'authentification JWT avec contrôle d'accès basé sur les rôles (RBAC — viewer/operator/admin) a été développée :

- POST `/auth/login` — authentification, retourne un token JWT (expiration 60 min)
- GET `/auth/me` — informations utilisateur courant
- Rate limiting sur `/auth/login` — 5 tentatives/minute par IP (anti brute-force, slowapi)
- CORS restreint aux origines du dashboard React (remplace l'ancien `allow_origins=["*"]`)

État du déploiement

Le code JWT/RBAC est finalisé (auth.py, main.py, seed_users.py). Le déploiement Docker rencontre un problème d'environnement en cours de résolution au moment de la rédaction de ce rapport ; la validation complète (tests curl 200/401/403/429) sera documentée en annexe dès résolution.

Détail des endpoints prévus et de leur niveau d'accès :

Endpoint	Méthode	Rôle requis	Description
<code>/auth/login</code>	POST	public	Authentification, retourne un JWT (5 tentatives/min max)
<code>/dashboard</code>	GET	viewer+	Résumé (injections 24h, alertes critiques, capteurs actifs)
<code>/readings/latest</code>	GET	viewer+	Dernière mesure par capteur et par champ
<code>/readings/history</code>	GET	viewer+	Historique 24h agrégé par capteur
<code>/alerts</code>	GET	operator+	Liste des 50 dernières alertes de fraude
<code>/admin/users</code>	GET	admin	Gestion des comptes utilisateurs

La hiérarchie des rôles est cumulative : un compte admin hérite des permissions operator et viewer. Cette granularité répond à l'objectif O5 du sujet, qui demande de justifier le rôle et les spécificités de chaque profil : le profil viewer correspond à un responsable de coopérative consultant les données de son site, operator à un superviseur régional accédant aux alertes de fraude, et admin à l'équipe technique gérant les comptes.

9. Dashboard React

Le dashboard React (cacao_frontend, port 5173) affiche la supervision temps réel des 4 coopératives avec plusieurs vues fonctionnelles observées :

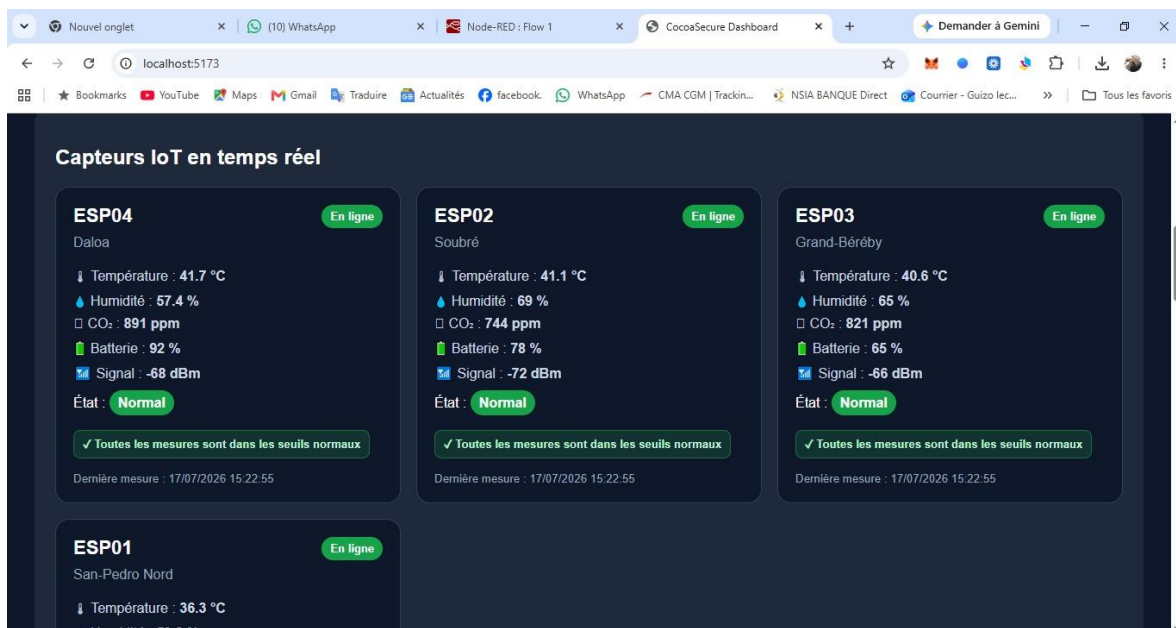


Figure 7 — Vue "Capteurs IoT en temps réel" : cartes ESP01-ESP04 avec température, humidité, CO₂, batterie, signal et statut.

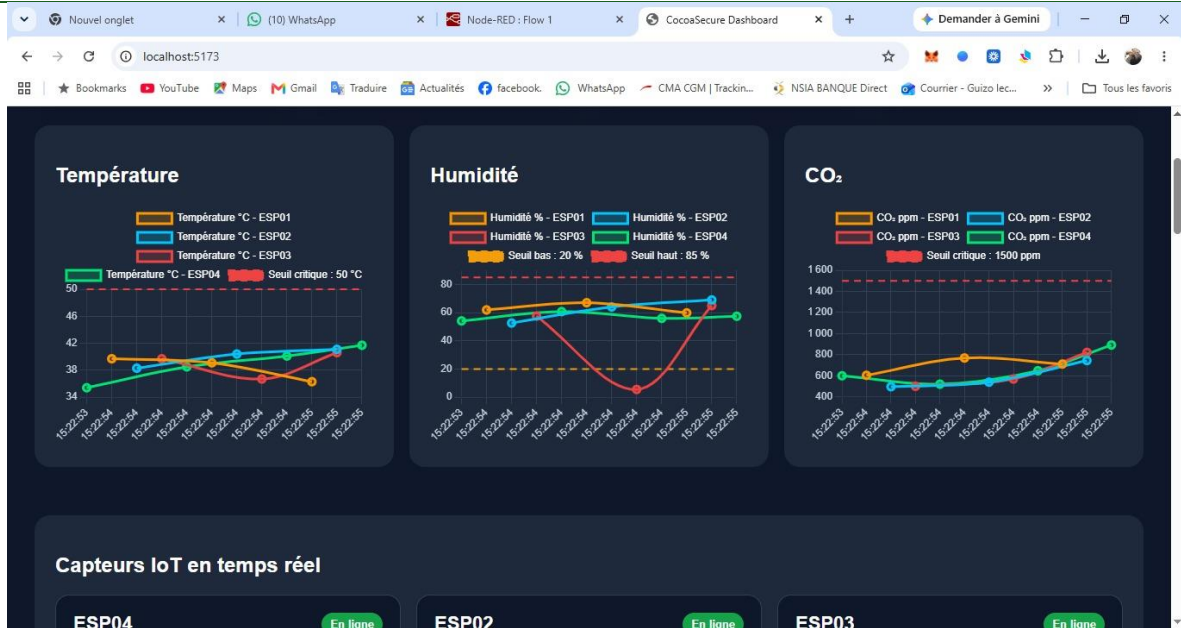
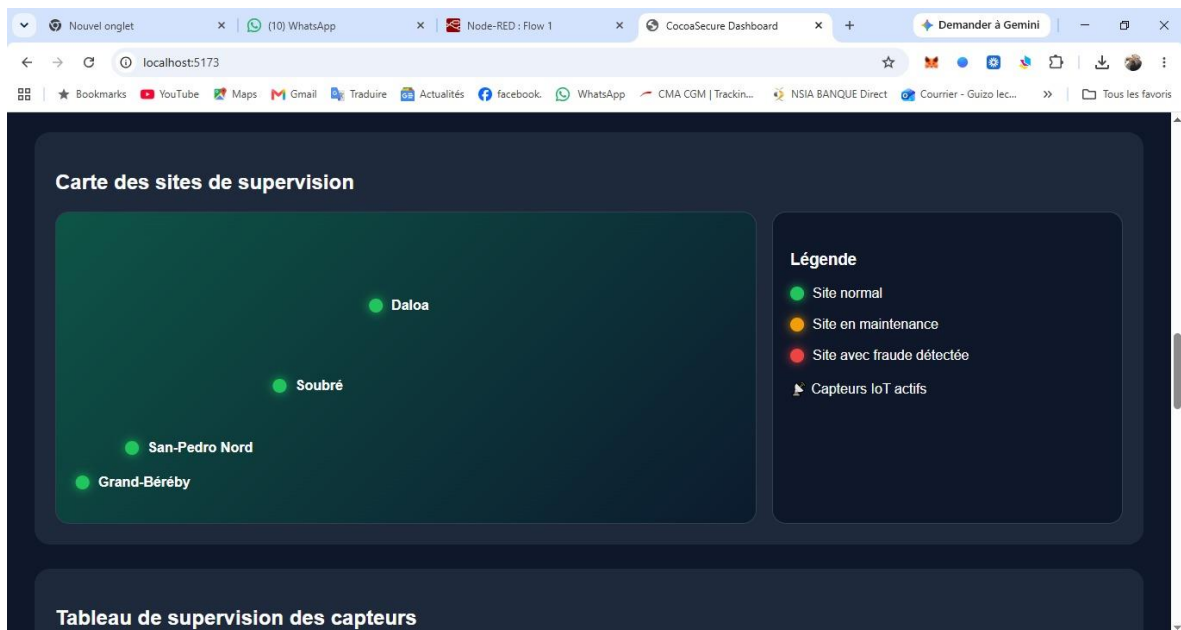
Figure 8 — Vue graphiques historiques : courbes Température / Humidité / CO₂ par capteur avec seuils critiques affichés.

Figure 9 — Carte de supervision géographique des 4 sites (San-Pedro Nord, Soubré, Grand-Béréby, Daloa) avec légende d'état.

Device	Site	Température	Humidité	CO ₂	Batterie	Signal	État
ESP04	Daloa	41.7 °C	57.4 %	891 ppm	92 %	-68 dBm	Normal
ESP02	Soubré	41.1 °C	69 %	744 ppm	78 %	-72 dBm	Normal
ESP03	Grand-Béréby	40.6 °C	65 %	821 ppm	65 %	-66 dBm	Normal
ESP01	San-Pedro Nord	36.3 °C	59.8 %	711 ppm	70 %	-77 dBm	Normal

Historique des dernières injections

Heure	Device	Site	Température	Humidité	CO ₂	État
-------	--------	------	-------------	----------	-----------------	------

Figure 10 — Tableau de supervision des capteurs, vue tabulaire consolidée.

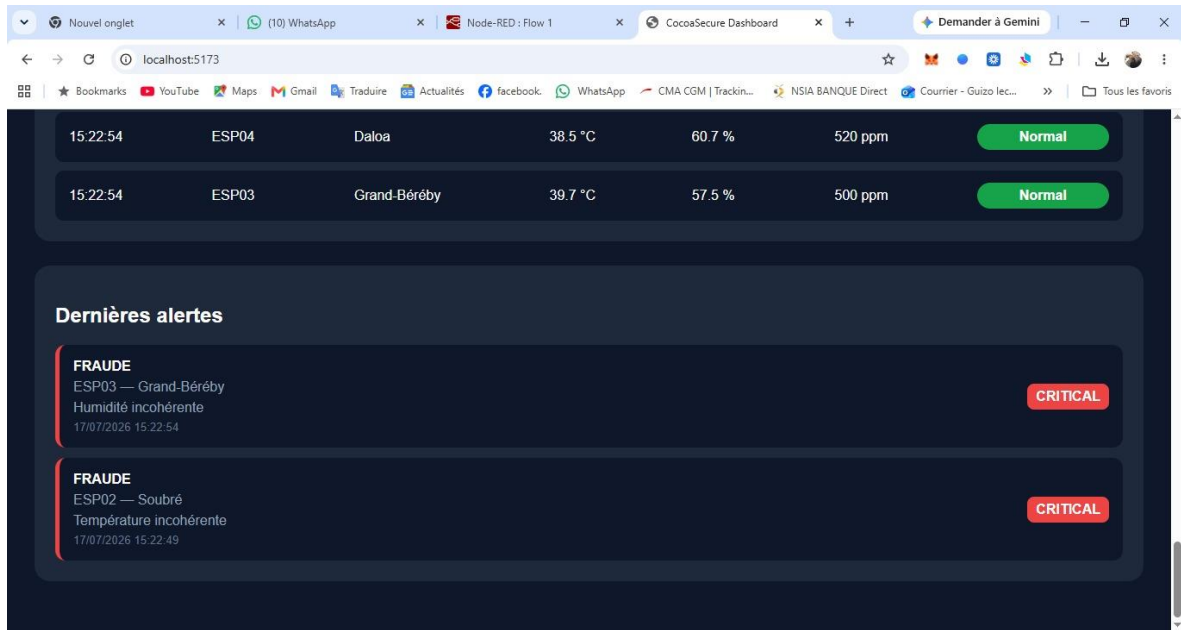
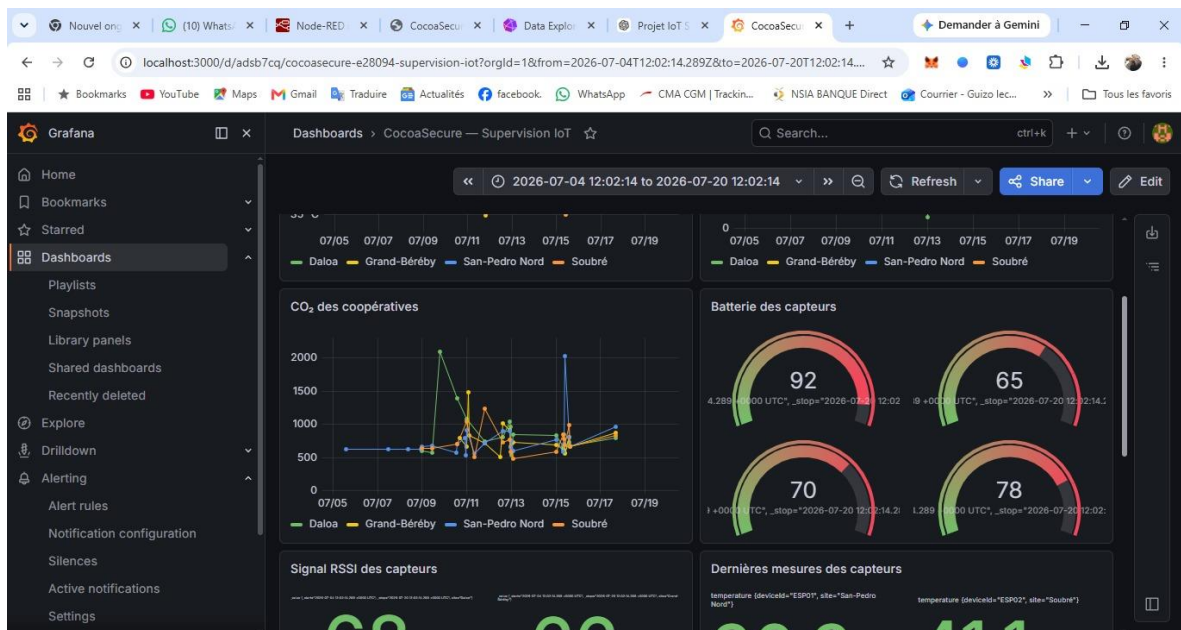


Figure 11 — Historique des dernières alertes de fraude (niveau CRITICAL), avec site et cause (humidité/température incohérente).

10. Stack d'observabilité

L'intégration Grafana (cacao_grafana, port 3000) est pleinement opérationnelle. Le dashboard "CocoaSecure — Supervision IoT" agrège les données InfluxDB des 4 coopératives sur une fenêtre configurable (16 jours dans la capture ci-dessous), avec plusieurs panels métier et techniques.

Figure 6b — Dashboard Grafana : évolution du CO₂ par coopérative et jauges de niveau de batterie des capteurs (San-Pedro Nord, Soubré, Grand-Béréby, Daloa).

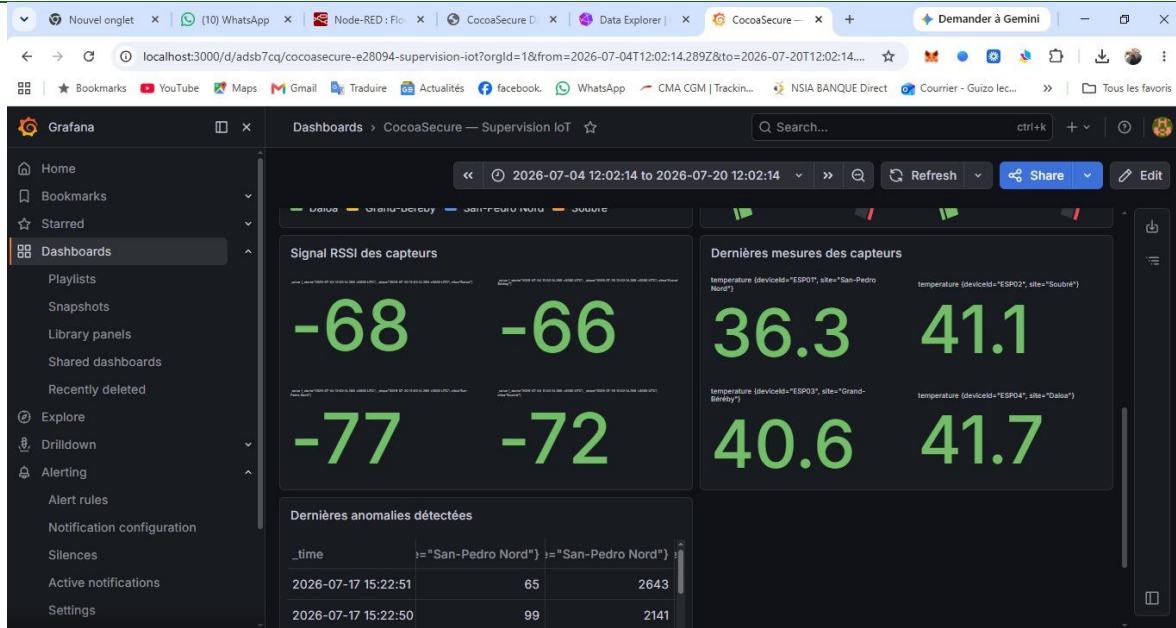


Figure 6c — Dashboard Grafana : signal RSSI par site, dernières mesures de température par capteur (deviceId/site en label), et tableau des dernières anomalies détectées avec horodatage.

Ce dashboard confirme l'intégration InfluxDB → Grafana pour les métriques IoT/métier (température, CO2, batterie, signal, anomalies), répondant à l'exigence C3 ("Grafana + au moins 1 autre outil opérationnel") et à l'exigence C4 d'intégration Grafana dans le dispositif de supervision.

InfluxDB Data Explorer permet en complément l'interrogation directe et ad-hoc des séries temporelles (voir Figure 6).

Reste à finaliser : export des dashboards Grafana au format JSON pour versioning dans le dépôt (observability/grafana/), et intégration d'un second outil d'observabilité (Kibana ou Prometheus) pour les logs et métriques système, conformément à l'exigence "au moins 2 outils" du sujet.

Métriques identifiées comme prioritaires pour le monitoring applicatif : taux de fraude détectée / 24h, latence de traitement Node-RED, nombre de capteurs actifs, cache hit rate Redis (une fois Sentinel opérationnel).

Métriques de sécurité à exposer via Prometheus/Grafana pour la détection en continu des attaques documentées en section 4 :

Métrique	Source	Seuil d'alerte
Connexions MQTT anonymes rejetées	Logs mosquito-secure	> 5/min → tentative d'intrusion
Messages hors seuils physiques	Node-RED fraudScore	> 10% du flux → campagne d'injection
Échecs d'authentification API	slowapi / logs FastAPI	> 5/min/IP → brute-force (A4)
Messages dupliqués (même payload)	MongoDB fraud_alerts	> 0 → suspicion de replay (A3)

Scénario de séquence — Attaque MITM (A2) vs. flux légitime

Le diagramme de séquence ci-dessous contraste le flux légitime (ESP32 authentifié) avec le flux d'attaque A2 (client non authentifié usurpant un deviceId) :

```
Flux légitime (attendu après sécurisation) :  ESP01 --[cert. client X.509]--> Broker:8883 --[TLS OK]--> Node-RED --> InfluxDB
Flux d'attaque A2 (observé avant sécurisation) :  Attaquant --[aucun cert., deviceId=ESP01 usurpé]--> Broker:1883 --[allow_anonymous=true : ACCEPTÉ]--> Node-RED
--[fraudScore élevé]--> MongoDB (fraud_alerts) + alerte dashboard
Flux d'attaque A2 rejoué (après sécurisation, port 8883) :  Attaquant --[aucun cert.]--> Broker:8883 --[tlsv1 alert unknown ca]--> REJETÉ
```

Ce contraste illustre la valeur ajoutée de la contre-mesure : la même tentative d'attaque, rejouée à l'identique, change de résultat uniquement grâce à la couche TLS/mTLS ajoutée entre les deux captures.

11. Conclusion, limites et perspectives

Ce rapport documente une plateforme IoT fonctionnelle de bout en bout, avec un volet sécurité particulièrement abouti : trois scénarios d'attaque réalistes ont été menés et documentés avec preuves reproductibles (Wireshark, logs), et la contre-mesure principale — TLS 1.3 avec authentification mutuelle X.509 — a été déployée et validée expérimentalement (connexion acceptée avec certificat, rejetée sans certificat).

Limites identifiées

- Redis déployé en instance unique, sans Sentinel — non-conforme à l'exigence HA du sujet (priorité de correction immédiate)
- JWT/RBAC développé mais déploiement Docker non finalisé au moment de la rédaction
- Un second outil d'observabilité (Kibana ou Prometheus) reste à intégrer aux côtés de Grafana
- Protection anti-replay applicative (nonce/TTL) non implémentée — le mTLS bloque l'usurpation réseau mais pas le vol physique d'un certificat déjà déployé

Perspectives immédiates

- Migrer Redis vers une configuration Sentinel 3 nœuds (1 master + 2 replicas + 3 sentinels)
- Finaliser le déploiement JWT et exécuter la suite de tests (200/401/403/429)
- Stabiliser l'intégration Grafana et exporter les dashboards
- Implémenter un mécanisme anti-replay (nonce horodaté, fenêtre de validité stricte)
- Réaliser des tests de charge (hey/wrk) avec et sans cache Redis pour objectiver le gain de performance

12. Répartition des rôles et contribution individuelle

Conformément à la structure pluridisciplinaire du projet (2 étudiants BD-GL + 1 étudiant RIST), les contributions se répartissent comme suit :

Rôle	Contribution documentée dans ce rapport
Étudiant BDGL/RIST (sécurité)	Analyse STRIDE complète (section 3), scénarios d'attaque A1/A2/A3 avec preuves (section 4), déploiement et validation TLS 1.3 + mTLS (section 5), diagrammes de séquence attaque (section 10)
Étudiants BD-GL (pipeline, cache, API, frontend)	Pipeline Node-RED (section 7), architecture InfluxDB/MongoDB, dashboard React 6 vues (section 9), déploiement Docker Compose (section 2), API REST (section 8)

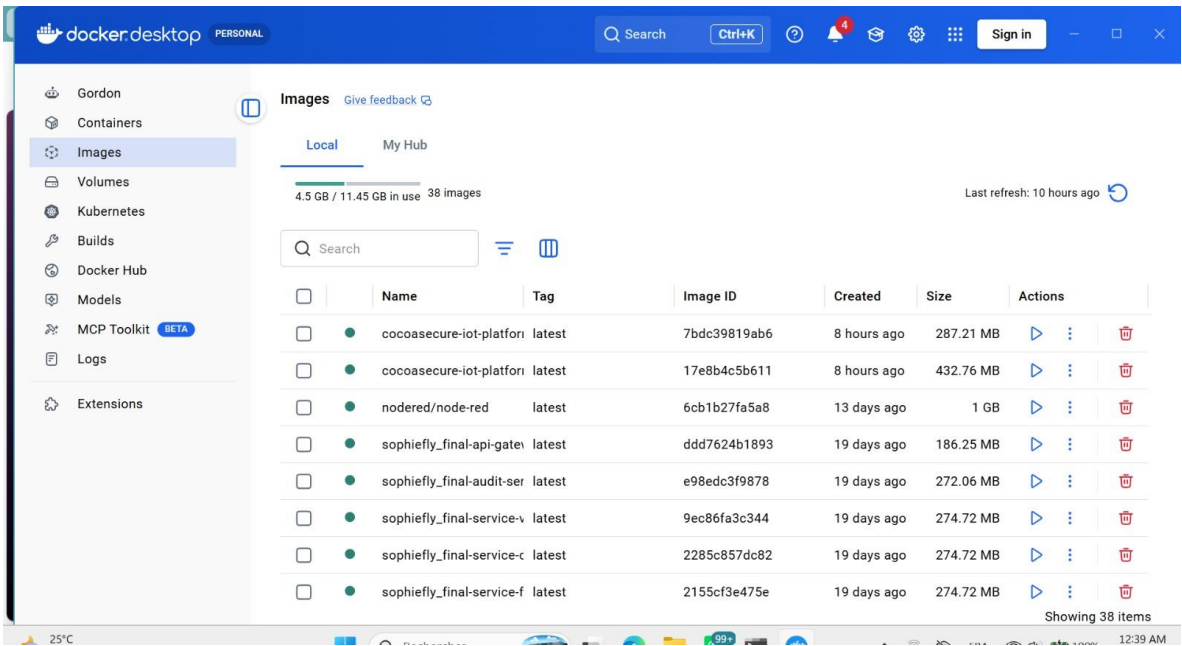


Figure 12 — Images Docker locales de la plateforme, confirmant la présence des builds cocoasecure-iot-platform (API et frontend) en plus des images tierces (Node-RED, Mosquitto, Redis, InfluxDB, MongoDB, Grafana).

13. Annexe — Tests et couverture

Les tests unitaires Pytest sur l'API FastAPI sont prévus pour couvrir au minimum 70% du code, conformément à l'objectif O5 du sujet, en priorité sur les modules d'authentification (auth.py) et les endpoints de lecture (dashboard, readings, alerts). L'exécution complète de la suite de tests et le rapport de couverture (coverage.py) seront fournis en complément dès la stabilisation du déploiement Docker de l'API sécurisée (voir section 8).

14. Références

- Redis Sentinel Documentation — redis.io/docs/management/sentinel
- Mosquitto TLS Configuration — mosquitto.org/man/mosquitto-conf-5.html
- FastAPI Security — fastapi.tiangolo.com/tutorial/security
- ARTCI — Cadre réglementaire IoT et données en Côte d'Ivoire — artci.ci
- Loi n°2013-451 relative à la lutte contre la cybercriminalité en Côte d'Ivoire